# Comprehensive Data Cleaning Report

## Retail Sales Capstone Project

---

## Executive Summary

This report documents the systematic data cleaning process applied to five datasets in the Retail Sales Analysis project. The cleaning process followed a standardized methodology across all datasets to ensure consistency, data quality, and analytical readiness.

**Datasets Processed:**

1. Sales Data (`sales_data.csv`)
2. Customers Data (`customers.csv`)
3. Products Data (`products.csv`)
4. Returns Data (`returns.csv`)
5. Stores Data (`stores.csv`)

---

## 1. Data Import and Initial Assessment

### Objective

Load raw data files and assess initial data quality across all datasets.

### Process Applied to All Datasets

**Sales Data:**

- Source: `Raw_data/sales_data.csv`
- Initial assessment conducted for missing values

**Customers Data:**

- Source: `Raw_data/customers.csv`
- Initial assessment conducted for missing values

**Products Data:**

- Source: `Raw_data/products.csv`
- Initial assessment conducted for missing values

**Returns Data:**

- Source: `Raw_data/returns.csv`
- Initial assessment conducted for missing values

**Stores Data:**

- Source: `Raw_data/stores.csv`
- Initial assessment conducted for missing values

**Key Action:** Missing value counts were printed for each dataset before any cleaning operations to establish baseline data quality metrics.

---

## 2. Column Name Standardization

### Objective

Ensure consistent column naming conventions across all datasets by removing whitespace.

### Process

Applied to **all five datasets** using `df.columns = df.columns.str.strip()`

### Rationale

- Removes leading and trailing whitespace from column headers
- Prevents referencing errors during data processing
- Ensures consistency across the entire data pipeline

### Datasets Affected

✓ Sales Data
✓ Customers Data
✓ Products Data
✓ Returns Data
✓ Stores Data

---

## 3. Missing Value Treatment

### Objective

Handle missing values appropriately based on field criticality and business logic.

### 3.1 Critical Fields - Row Deletion

**Fields where missing values resulted in row removal:**

| Dataset | Critical Fields | Action |
|---|---|---|
| Sales | store_id | Dropped rows with missing store_id |
| Customers | customer_id, age, signup_date | Dropped rows with missing values |
| Products | product_id, cost_price, unit_price | Dropped rows with missing values |
| Returns | return_id, order_id, return_date | Dropped rows with missing values |
| Stores | store_id, operating_cost | Dropped rows with missing values |

**Rationale:** These fields are essential for data integrity and cannot be reliably imputed. Records without these identifiers cannot be used in analysis.

### 3.2 Non-Critical Fields - Imputation

**Fields where missing values were filled:**

| Dataset | Field | Imputation Method |
|---|---|---|
| Customers | `first_name`, `last_name` | Filled with 'Unknown' |
| Customers | `gender` | Filled with 'Not Specified' |
| Customers | `region` | Filled with 'Unknown' |
| Products | `product_name` | Filled with 'Unknown Product' |
| Products | `category` | Filled with 'Uncategorized' |
| Products | `brand` | Filled with 'Unknown Brand' |
| Products | `margin_pct` | Filled with 0 |
| Returns | `return_reason` | Filled with 'Not Specified' |
| Stores | `store_name` | Filled with 'Unknown Store' |
| Stores | `store_type` | Filled with 'Not Specified' |
| Stores | `region` | Filled with 'Unknown Region' |
| Stores | `city` | Filled with 'Unknown City' |
| Stores | `operating_cost` | Filled with median value |

**Rationale:** These fields provide supplementary information and can be safely imputed without compromising data integrity.

---

## 4. Data Type Standardization

### Objective

Convert data columns to appropriate data types for proper analysis and calculations.

### 4.1 Datetime Conversions

**Process:** Used `pd.to_datetime()` with `errors='coerce'` parameter

| Dataset | Field | Conversion |
|---|---|---|
| Sales | `order_date` | Converted to datetime |
| Customers | `signup_date` | Converted to datetime |
| Returns | `return_date` | Converted to datetime |

**Rationale:** Enables time-series analysis, date-based filtering, and temporal pattern identification.

### 4.2 Numeric Conversions

**Process:** Used `pd.to_numeric()` with `errors='coerce'` parameter

| Dataset | Fields Converted |
|---|---|
| Sales | `quantity`, `unit_price`, `discount_pct`, `total_amount` |
| Customers | `age` |
| Products | `cost_price`, `unit_price`, `margin_pct` |
| Stores | `operating_cost` |

**Rationale:** Ensures mathematical operations can be performed correctly and prevents type-related errors in calculations.

**Error Handling:** The `errors='coerce'` parameter converts invalid values to NaN, allowing graceful handling of data quality issues.

---

## 5. Text Data Standardization

### Objective

Ensure consistent formatting of text fields across all datasets.

### Process Applied

**String Cleaning Operations:**

1. Remove leading/trailing whitespace: `.str.strip()`
2. Standardize capitalization: `.str.title()`

### 5.1 Fields Standardized by Dataset

**Customers Data:**

- first_name
- last_name
- gender
- region

**Products Data:**

- product_name
- category
- brand

**Returns Data:**

- return_reason

**Stores Data:**

- store_name
- store_type
- region
- city

### 5.2 Value Mapping and Standardization

**Gender Standardization (Customers):**

```
Mapping Applied:
'M' → 'Male'
'F' → 'Female'
'Other' → 'Other'
'Not Specified' → 'Not Specified'
```

**Store Type Standardization (Stores):**

```
Mapping Applied:
'Shopping Mall' → 'Mall'
'Stand-Alone' → 'Standalone'
```

**Return Reason Standardization (Returns):**

```
Mapping Applied:
'Damaged', 'Faulty' → 'Defective'
'Delayed' → 'Late Delivery'
'Incorrect Item' → 'Wrong Item'
```

**Rationale:** Standardization enables accurate grouping, consistent reporting, and eliminates variations caused by data entry inconsistencies.

---

## 6. Duplicate Removal

### Objective

Identify and remove duplicate records to ensure data integrity.

### 6.1 Exact Duplicate Removal

**Process:** Applied `df.drop_duplicates()` to all datasets

| Dataset | Action |
|---|---|
| Sales | Removed exact duplicate rows |
| Customers | Removed exact duplicate rows |
| Products | Removed exact duplicate rows |
| Returns | Removed exact duplicate rows |
| Stores | Removed exact duplicate rows |

### 6.2 ID-Based Duplicate Removal

**Process:** Removed duplicates based on primary identifier fields

| Dataset | Identifier Field | Method |
|---|---|---|
| Returns | `return_id` | Kept first occurrence |
| Stores | `store_id` | Kept first occurrence |

**Rationale:**

- Exact duplicates inflate record counts and skew analysis

- ID-based duplicates indicate data entry errors or system issues
- Each unique identifier should appear only once in the dataset

**Verification:** Duplicate counts were printed before and after removal for all datasets.

---

## 7. Outlier Removal and Data Validation

### Objective

Remove unrealistic, invalid, and outlier values based on business logic.

### 7.1 Sales Data Validation

**Rules Applied:**

- `total_amount > 0` (Sales must have positive values)
- `quantity > 0` (Must include at least one item)
- `unit_price > 0` (Products must have positive prices)

### 7.2 Customers Data Validation

**Rules Applied:**

- `age > 0` (Age must be positive)
- `age <= 120` (Age must be realistic)

### 7.3 Products Data Validation

**Rules Applied:**

- `cost_price > 0` (Cost must be positive)
- `unit_price > 0` (Price must be positive)
- `unit_price >= cost_price` (Selling price should cover cost)
- `margin_pct >= 0` (Margin cannot be negative)
- `margin_pct <= 100` (Margin cannot exceed 100%)

### 7.4 Returns Data Validation

**Rules Applied:**

- `return_date <= current_date` (No future return dates)

### 7.5 Stores Data Validation

**Rules Applied:**

- `operating_cost > 0` (Cost must be positive)
- `operating_cost <= 1,000,000` (Reasonable upper limit)

**Rationale:** These business rules ensure data reflects realistic scenarios and removes data entry errors or system anomalies.

---

## 8. Derived Column Creation

### Objective

Create calculated fields to enhance analytical capabilities.

### 8.1 Customers Data - Derived Columns

**Column Created:** `age_group`

**Definition:** Age segmentation into meaningful groups

```
Bins: [0, 18, 25, 35, 45, 55, 65, 120]
Labels: ['<18', '18-25', '26-35', '36-45', '46-55', '56-65', '65+']
```

**Purpose:** Enables demographic analysis and customer segmentation.

### 8.2 Products Data - Derived Columns

**Column Created:** `price_category`

**Definition:** Product price segmentation

```
Bins: [0, 20, 50, 100, 500, 10000]
Labels: ['Budget', 'Economy', 'Mid-Range', 'Premium', 'Luxury']
```

**Purpose:** Facilitates pricing strategy analysis and product positioning.

### 8.3 Returns Data - Derived Columns

**Columns Created:**

- `return_year` - Year of return
- `return_month` - Month number
- `return_month_name` - Month name

- `return_quarter` - Quarter (Q1-Q4)

**Purpose:** Enables temporal analysis of return patterns and seasonality trends.

### 8.4 Stores Data - Derived Columns

**Column Created:** `cost_category`

**Definition:** Operating cost segmentation

```
Bins: [0, 50000, 100000, 200000, 500000, 10000000]
Labels: ['Very Low', 'Low', 'Medium', 'High', 'Very High']
```

**Purpose:** Enables cost-based store classification and resource allocation analysis.

---

## 9. Data Export and Storage

### Objective

Save cleaned datasets for downstream analysis.

### Output Files Generated

| Original File | Cleaned File | Location |
|---|---|---|
| `sales_data.csv` | `sales_cleaned.csv` | `CLeaned_data/` |
| `customers.csv` | `customers_cleaned.csv` | `CLeaned_data/` |
| `products.csv` | `products_cleaned.csv` | `CLeaned_data/` |
| `returns.csv` | `returns_cleaned.csv` | `CLeaned_data/` |
| `stores.csv` | `stores_cleaned.csv` | `CLeaned_data/` |

**Export Parameters:**

- Format: CSV
- Index: Excluded (`index=False`)
- Encoding: UTF-8 (default)

---

## 10. Data Quality Metrics Summary

**Quality Checks Performed Across All Datasets**

✓ No missing values in critical identifier fields
✓ All duplicate records removed
✓ All date fields converted to datetime format
✓ All numeric fields validated and converted
✓ All text fields standardized (case and spacing)
✓ All categorical values mapped to standard terms
✓ All outliers removed based on business rules
✓ Derived columns created for enhanced analysis

**Dataset-Specific Validations**

**Sales Data:**

- All amounts, quantities, and prices are positive
- All orders have valid store identifiers

**Customers Data:**

- All ages are within realistic range (0-120)
- All customers have valid identifiers
- Gender values standardized

**Products Data:**

- Unit prices are greater than or equal to cost prices
- Margin percentages are between 0-100%
- All products have valid identifiers

**Returns Data:**

- No future return dates
- Return reasons standardized
- Temporal columns created for analysis

**Stores Data:**

- Operating costs within reasonable range
- Store types standardized
- Cost categories created for classification

---

## 11. Technical Implementation Summary

### Python Libraries Used

- **pandas** - Data manipulation and cleaning

### Key Functions Applied

| Function | Purpose | Datasets Applied |
|---|---|---|
| `pd.read_csv()` | Data import | All |
| `str.strip()` | Remove whitespace | All |
| `dropna()` | Remove missing values | All |
| `fillna()` | Impute missing values | Customers, Products, Returns, Stores |
| `pd.to_datetime()` | Date conversion | Sales, Customers, Returns |
| `pd.to_numeric()` | Numeric conversion | Sales, Customers, Products, Stores |
| `drop_duplicates()` | Remove duplicates | All |
| `replace()` / `map()` | Value standardization | Customers, Products, Returns, Stores |
| `pd.cut()` | Create categorical bins | Customers, Products, Stores |
| `to_csv()` | Data export | All |

---

## 12. Data Cleaning Workflow Summary

### Standardized Cleaning Pipeline

```
1. Import Data
  ↓
2. Assess Missing Values
  ↓
3. Clean Column Names
  ↓
4. Handle Missing Values (Drop or Fill)
  ↓
5. Standardize Data Types
```

↓
6. Standardize Text Fields
↓
7. Remove Duplicates
↓
8. Remove Outliers
↓
9. Create Derived Columns
↓
10. Export Cleaned Data

This pipeline was consistently applied across all five datasets to ensure standardization and reproducibility.

---

## 13. Data Integrity Verification

**Post-Cleaning Validation Performed**

**For Each Dataset:**

1. Verified no duplicates remain
2. Confirmed all critical fields are non-null
3. Validated data type conversions
4. Checked value ranges for numeric fields
5. Confirmed text standardization
6. Verified derived column calculations

**Console Output Tracking:**

- Missing values before cleaning
- Row counts at each stage
- Duplicate counts before and after removal
- Final row counts after outlier removal

---

## 14. Recommendations for Analysis

**Data is Now Ready For:**

1. **Exploratory Data Analysis (EDA)**
    - Statistical summaries

- Distribution analysis
- Trend identification

2. **Advanced Analytics**
    - Customer segmentation analysis
    - Product performance analysis
    - Sales trend analysis
    - Return pattern analysis
    - Store performance evaluation
3. **Data Integration**
    - Cross-table joins for comprehensive analysis
    - Customer purchase behavior analysis
    - Product profitability analysis
    - Store efficiency metrics
4. **Visualization**
    - Time-series visualizations
    - Category-based comparisons
    - Geographical analysis
    - Performance dashboards

---

## 15. Conclusion

The data cleaning process successfully transformed five raw datasets into analysis-ready cleaned datasets. The standardized methodology ensured consistency across all datasets while addressing dataset-specific requirements.

**Key Achievements:**

- ✓ Systematic handling of missing values
- ✓ Consistent data type standardization
- ✓ Comprehensive duplicate removal
- ✓ Business-logic-based outlier treatment
- ✓ Enhanced analytical capability through derived columns
- ✓ Maintained data integrity throughout the process

**Final Status:** All datasets are clean, validated, and ready for comprehensive retail sales analysis.

---

## Appendix: Cleaning Statistics

### Data Volume Summary

| Dataset | Original Records | Final Records | Retention Rate |
|---|---|---|---|
| Sales | [From console] | [From console] | [Calculated] |
| Customers | [From console] | [From console] | [Calculated] |
| Products | [From console] | [From console] | [Calculated] |
| Returns | [From console] | [From console] | [Calculated] |
| Stores | [From console] | [From console] | [Calculated] |

**Note:** Actual numbers are logged in console output during execution.

---

**Report Prepared By:** Mohammed Niyaz Ali
**Project:** Retail Sales Capstone Analysis
**Data Location:** C:\Users\Niyaz Ali\Documents\Retails_sales_capstone\Data\